

杂题选讲

徐骁扬

前言

从成功落选国家队之后就没怎么做过 OI 题了。

打 OI 的时候做的题基本上 60% 是联考相关的，40% 是大家都会做的题。

看来只能从 ACM 比赛里找点像 OI 题的题了。感觉都是一些比较好玩（存疑）的 trick。

题目没有按照难度排序。

简要题意

Dumb Problem

给定一个长度为 n 的序列 a_n ，你可以选择一个排列 $p_1, \dots, p_n$ ，依次对 $i = 1, 2, \dots, n$ 进行操作：令 $a_{p_i} \leftarrow a_{p_{i-1}} + a_{p_i} + a_{p_{i+1}}$ 。认为 $a_0 = a_{n+1} = 0$ 。

问操作之后 $\sum_{i=1}^n a_i$ 最大为多少。

$1 \leq n \leq 2 \times 10^5$, $|a_i| \leq 10^8$ 。1s, 1G。

Hint

- 一个 a_i 对答案的贡献是什么。
- 简化 “选择一个排列 p ”。

分析题意

对于 a_i , 它的值会“蔓延”到 a_{i-1} 和 a_{i+1} 。

如果 i 在排列中的位置在 $i-1$ 之前, 则 a_i 会再次蔓延到 a_{i-2} 和 a_i , 依此类推。

实际上我们只关注所有相邻 i 和 $i+1$ 在序列中出现的相对顺序。

我们称 $<$ 为 i 在 $i+1$ 之前出现, $>$ 为 i 在 $i+1$ 之后出现。

求解答案

对于一整个 $<$ 或 $>$ 的连续段的转移需要一次性计算，所以尝试使用动态规划求解：

记 $f_{i,0/1}$ 表示 $i-1 < i$ 或 $i-1 > i$ 的情况下，已经计算过的连续段的贡献和最大是多少。

对于 $j < i$ ，我们有转移：

$$f_{i,0} \leftarrow f_{j,1} + \sum_{k=j+1}^i (2(i-k) + 3)a_k + (i-j)a_j + (i-j)a_{j-1}。$$

$$f_{i,1} \leftarrow f_{j,0} + \sum_{k=j}^{i-1} (2(k-j) + 3)a_k + (i-j)a_i + (i-j)a_{i+1} + (a_i - a_j)。$$

其中 $a_i - a_j$ 是由于顶点会被多计算依次，底点会被少计算一次。

优化求解

令 $s_i = \sum_{j=1}^i a_j$, $s'_i = \sum_{j=1}^i ia_j$, 则可以将上页转移中的求和式去掉:

$$f_{i,0} \leftarrow f_{j,1} - 2(s'_i - s'_j) + (2i+3)(s_i - s_j) + (i-j)a_j + (i-j)a_{j-1}.$$

$$f_{i,1} \leftarrow f_{j,0} + 2(s'_{i-1} - s'_{j-1}) - (2j+3)(s_{i-1} - s_{j-1}) + (i-j+1)a_i + (i-j)a_{i+1}.$$

发现分别只有 $i(-2s_j + a_j + a_{j-1})$ 和 $j(-2s_{i-1} - a_i - a_{i+1})$ 这两项是和 i, j 都有关的, 其余要么只与 i 有关, 要么只与 j 有关。

因此转移可是使用李超线段树来进行优化, 最终时间复杂度 $O(n \log n)$ 。

简要题意

Bipartite Graph Weighting Problem

给定一张 $2n$ 个点, m 条边的二分图 G , 左部和右部各 n 个点。有 q 次询问, 每一次询问给定三个数 a, b, k , 对于每一次询问:

初始每一个点各有一个初始为 0 的点权, 并进行如下操作若干次:

- ① 选择一个左部点集的子集 S (可以为空), 以及一个**正实数** p ;
- ② 将 S 中的点的点权增加 $a \times p$;
- ③ 将邻域 $N(S)$ 中的点点权增加 $b \times p$ 。

问在所有操作的 p 之和小于等于 1, 所有点点权和小于等于 k 的情况下, 左部点点权和最大为多少。

$1 \leq n \leq 2000$, $0 \leq m \leq 10^4$, $1 \leq q \leq 2 \times 10^5$ 。2s, 1G。

Hint

- 什么样的 $(S, N(S))$ 会被选择。
- 最优的操作会长什么样子。
- 如何得到所有需要的 $(S, N(S))$ 。

分析操作

可以将 $a \times p$ 和 $b \times p$ 先都当作 p 处理，在最终求和的时候再考虑 a 和 b 的影响。

同时，对于每一个点单独维护点权是没有必要的，只需要维护左部点点权和 X 和右部点点权和 Y 两个值即可。

假设此时对 S 和 p 操作，则就是给 X 增加 $p|S|$ ，并给 Y 增加 $p|N(S)|$ 。

由于 S 可以为空，将限制加强为 $\sum p = 1$ 不影响结果。

那么将所有 $(|S|, |N(S)|)$ 看作是笛卡尔坐标系上的点集 V ，则最终能够构造出的 (X, Y) 一定在 V 的凸包内。

计算答案

那么问题就变成，在满足 $aX + bY \leq k$ 的情况下， aX 最大为多少。

如果 $aX + bY = k$ 和凸包有交，那么最优的点就是 $aX + bY = k$ 和凸包边界靠右的交点。

因此如果能够求解出 V 的右下凸包，则可以直接通过二分答案来求解单个询问的答案。

求解凸包

将所有可能的 $(|S|, |N(S)|)$ 找出来是不现实的，我们需要找在下半凸壳上的 $(|S|, |N(S)|)$ 还能有什么性质。

对于在右下凸壳上的点 $(|S|, |N(S)|)$ ，一定存在一个非负实数 V ，使得 $|N(S)| - v|S|$ 是最小的。

有一个分治求解凸包的 trick：假设已知 (x_1, y_1) 和 (x_2, y_2) ($x_1 < x_2, y_1 < y_2$) 是右下凸壳上的点，那么考虑求解 $(x_2 - x_1)y - (y_2 - y_1)x$ 最小的点。

如果 (x, y) 不等于 (x_1, y_2) 或 (x_2, y_2) ，则说明 (x, y) 也是凸包上的点。然后可以递归到 $(x_1, y_1), (x, y)$ 和 $(x, y), (x_2, y_2)$ 两部分进行求解。

时间复杂度为 $O(\alpha f)$ ，其中 α 为右下凸壳上点的数量， f 为求解单个 (x, y) 的时间复杂度。

求解 $\arg \min(By - Ax)$

由于 $B|N(S)| - A|S| = B|N(S)| + A|CS| - nA$ 。那么我们就是要最小化 $B|N(S)| + A|CS|$ 。

故问题等价于选择一个左部点的权为 A ，选择一个右部点的权为 B ，求最小权点覆盖。

使用网络流求解。

源点向左部点连容量为 A 的边，左部点向有连边的右部点连容量为 $+\infty$ 的边，右部点向汇点连容量为 B 的边。原图上最小权点覆盖等价于这个图上的最小割。

时间复杂度

由于整点凸包只会至多有 $O(n^{\frac{2}{3}})$ 个点, 因此最终时间复杂度为 $O(n^{\frac{2}{3}} \times \text{flow}(n, m))$, 其中 $O(\text{flow}(n, m))$ 为网络流的复杂度。经过实测可以通过该题。

简要题意

Sum and Product

定义 $f(n, k)$ 为满足以下条件的数对 a, b 的数量：

- $0 \leq a, b < n$;
- $k(a + b) \equiv ab \pmod{n}$ 。

T 组数据，给定 n, m ，求 $\sum_{k=0}^m f(n, k)$ ，答案对 998244353 取模。

$1 \leq n \leq 10^{14}$, $0 \leq m < n$, $1 \leq T \leq 10^4$ ，至多只有 10 组 $n \geq 10^8$ 。1s, 1G。

Hint

- 简化等式的限制，将加乘混合的式子变成更简单的形式。
- 中国剩余定理。

简化限制

如果 $k(a + b) \equiv ab \pmod{n}$, 则 $(a - k)(b - k) \equiv k^2 \pmod{n}$ 。

由于 a, b 是在 $\mathbb{Z}_n$ 中任取的, 令 $a \leftarrow a - k$, $b \leftarrow b - k$ 。

则 $f(n, k)$ 等于满足以下条件的数对 a, b 的数量: $ab \equiv k^2 \pmod{n}$, $a, b \in \mathbb{Z}_n$ 。

简化限制

记 $K = k^2 \bmod n$, $d = \gcd(K, n)$, $K = td$ 。

由于 t 和 n 互质。则对于数对 (a, b) 满足 $ab \equiv d \pmod{n}$, 都唯一对应着数对 $(a' = a, b' = tb)$ 使得 $a'b' \equiv K \pmod{n}$ 。

因此我们只需要考虑对于 n 的所有因子 d 求解 $ab \equiv d \pmod{n}$ 的数量 $f'(n, d)$ 即可。

计算答案

将 n 进行唯一分解 $n = \prod_{i=1}^t p_i^{\alpha_i}$, 则 $ab \equiv d \pmod{n}$ 等价于对于每一个 i , 有 $ab \equiv d \pmod{p_i^{\alpha_i}}$ 。

重复一次上页的推导, 我们实际上就是要求满足以下条件的数对 (a, b) 的数量 $g(i, k)$:

- $a, b \in \mathbb{Z}_{p_i^{\alpha_i}};$
- $ab \equiv p^k \pmod{p_i^{\alpha_i}}.$

计算答案

对于 $0 \leq k < \alpha_i$ 的情况，枚举 a 中 p_i 的指数，则有：

$$g(i, k) = \sum_{j=0}^k \varphi(p^{\alpha_i-j}) p^j = (k+1) \varphi(p^{\alpha_i}).$$

我们知道 $f' \left(n, \prod_{i=1}^t p_i^{a_i} \right) = \prod_{i=1}^t g(i, a_i)$ ，可以直接计算。

现在的问题变成了有多少个 $k \in [0, m]$ 满足 $\gcd(k^2, n) = d$ 。

计算答案

我们知道 $\gcd(k^2, n) = \gcd(\gcd(k, n)^2, n)$ 。

所以先考虑计算有多少个 $k \in [0, m]$ 满足 $\gcd(k, n) = d$ 。

有多少个 $k \in [0, m]$ 满足 $d \mid \gcd(k, n)$ 是好求的： $\left\lfloor \frac{m}{d} \right\rfloor + 1$ 。

然后对每一个质数维进行差分即可（等价于莫比乌斯容斥）。

单次询问的最终时间复杂度为 $O(\sqrt{n} + d(n) \log n)$ 。

简要题意

Many Approaches

有 N 个格子排成一排，从左到右依次编号为 $0, 1, \dots, N-1$ ，同时有 N 个人编号为 $0, 1, \dots, N-1$ 。对于一个序列 $X = (X_1, X_2, \dots, X_{|X|})$ ，定义一个 march 如下：

- ① 对于 $i = 0, 1, \dots, N-1$ ，让第 i 个人站在第 i 个格子。
- ② 对于 $j = 1, 2, \dots, |X|$ ，让每一个不在 X_j 的人向 X_j 移动一格。

简要题意

Many Approaches

给定一个长度为 M 的, 由 $0 \sim N-1$ 中的数字组成的序列。有 Q 次**在线**询问 (t, L, R, P) 。

- ① $t = 0$ 时, 问对序列 $(A_L, A_{L+1}, \dots, A_R)$ march 后, 第 P 个人在哪里。
- ② $t = 1$ 时, 问对序列 $(A_L, A_{L+1}, \dots, A_R)$ march 后, 第 P 个格子上有多少个人。

$1 \leq N, M, Q \leq 2 \times 10^5$ 。3s, 1G。

Hint

- 对于任意 march 过程中，人两两间的相对关系。
- 如何数学地描述 march 的过程。
- 很刻板的数据结构题。

解析操作

显然, 对于 $i < j$, 最终所在的位置必然有 $P_i \leq P_j$ 。

题目的目标是维护函数 $f_{[L,R]}$ 以及它的“逆” $f_{[L,R]}^{-1}$ 。表示:

第 x 个人在经过 $[L, R]$ 的序列之后会移动到 $f_{[L,R]}(x)$; 最终在 x 的编号最大的人为 $f_{[L,R]}^{-1}(x)$ 。

由于有 $f_{[L,R]}(x) = f_{[mid+1,R]}(f_{[L,mid]}(x))$, 故可以尝试使用线段树维护。

解析操作

令 $len := (R - L + 1)$ 。

对于的 $x \rightarrow -\infty$, 有 $f_{[L,R]}(x) = x + len$; $x \rightarrow +\infty$, 有 $f_{[L,R]}(x) = x - len$ 。
因此这个函数几乎和 $f(x)$ 一致, 只会有的变化, 又因为 $f_{[L,R]}$ 是单调的, 所以
可以尝试维护 $f_{[L,R]}(x) - x$ 的差分数组。

即维护一个大小为 $2len$ 的序列 $x_1, x_2, \dots, x_{2len}$, 使得

$$f_{[L,R]}(x) = x + len - \sum_{i=1}^{2len} [x \geq x_i].$$

那么它的逆就有 $f_{[L,R]}^{-1}(x) = x - len + \sum_{i=1}^{2len} [x - len + i \geq x_i]$

容易发现在经过适当处理之后, 可以 $O(\log n)$ 得求出 f 或者 f^{-1} 。

维护操作

现在的问题转变为合并两个 f 。问题的实质就是合并两个序列 x_1 和 x_2 。
我们有：

$$\begin{aligned} f_2(f_1(x)) &= f_1(x) - len_2 + \sum_{i=1}^{2len_2} [f_1(x) \geq x_{2i}] \\ &= x - (len_1 + len_2) + \sum_{i=1}^{2len_1} [x \geq x_{1i}] + \sum_{i=1}^{2len_2} [x \geq f_1^{-1}(x_{2i} - 1) + 1] \end{aligned}$$

实际上让每一个 x_2 对 x_1 求一遍逆即可。

维护操作

对于单个操作 A_i , 序列即为 $A_i, A_i + 1$ 。

考虑预处理的时间复杂度 $f(n) = 2f\left(\frac{n}{2}\right) + O(n \log n)$, 则 $f(n) = O(n \log^2 n)$ 。

对于单次询问的时间复杂度为 $O(\log n) \times O(\log n) = O(\log^2 n)$ 。

故总时间复杂度为 $O(n \log^2 n)$, 可以通过。

简要题意

Grammy's Kingdom

有 n 座车站编号为 $1, 2, \dots, n$, 其中有 m 座车站有机场。

如果你在第 i 个车站：若第 i 和 $i+1$ 个车站都没有被摧毁，则可以移动到第 $i+1$ 个车站；如果存在 $i \leq j$ 且第 i 和 j 座车站均有机场且未被摧毁，则可以移动到第 j 座车站。

记 G 为有多少对 $i < j$, 满足 i 可以经过若干步移动到 j 。

在接下来的 n 个时刻，每一个时刻都会有随机一座未被摧毁的车站被摧毁，设第 i 个时刻之后的 G 值为 G_i 。

求 $\sum_{i=0}^n E[G_i]$ 对 998244353 取模的结果。

$1 \leq n \leq 2 \times 10^6, 0 \leq m \leq n, 1s, 0.5G$ 。

Hint

- 对于一对固定的 (i, j) , 最优的路径是什么。
- $E[X + Y] = E[X] + E[Y]$ 。

分析路径

对于两个点 i, j , 如果它们之间没有机场, 则必然是从 i 直接移动到 j 。如果它们之间有机场, 则 i 至少要移动到其右侧第一个机场, j 至少要从其左侧第一个机场开始移动。这些点是必须经过的, 且这些点恰好构成一条从 i 到 j 的路径。故 i 能够到达 j , 当且仅当 i 到其右侧第一个机场以及 j 到其左侧第一个机场这些点都没有被摧毁。

计算贡献

由于期望的线性性，我们考虑一个需要保证 k 个点未被摧毁的路径会对答案做出多少的贡献。

也就是问 i 时刻，某确定的 k 个点没有被删除的概率是多少，即 $\sum_{i=0}^{n-k} \frac{\binom{n-k}{i}}{\binom{n}{i}}$ 。

计算贡献

$$\begin{aligned}\sum_{i=0}^{n-k} \frac{\binom{n-k}{i}}{\binom{n}{i}} &= \sum_{i=0}^{n-k} \frac{(n-k)!(n-i)!}{n!(n-k-i)!} \\&= \frac{(n-k)!}{n!} \sum_{i=0}^{n-k} \frac{(n-i)!}{(n-k-i)!} \\&= \frac{(n-k)!}{n!} \sum_{i=k}^n \frac{i!}{(i-k)!} \\&= \frac{(n-k)!k!}{n!} \sum_{i=k}^n \binom{i}{k} \\&= \frac{(n-k)!k!}{n!} \binom{n+1}{k+1} = \frac{n+1}{k+1}.\end{aligned}$$

计算贡献

问题被完全转化成：对于 $k = 1 \dots, n$ ，问有多少对点 $i < j$ 需要保证未被摧毁的点数为 k 。

对于两个机场之间的 t 个点，它们会对 $2 \leq i \leq t$ 产生 $t - i + 2$ 的贡献。

对于一个机场左侧（含这个机场）的 A 个点，和这个机场右侧（含这个机场）

的 B 个点，它们会对 i 产生：

$$\begin{cases} i, & 2 \leq i \leq \min(A, B) \\ \min(A, B), & \min(A, B) < i \leq \max(A, B) \\ A + B - i, & \max(A, B) < i \leq A + B \end{cases}$$

贡献。

对于一个机场左侧（含这个机场）的 A 个点，以及另一个机场右侧（含这个机场）的 B 个点，它们会对 i 产生：

$$\begin{cases} i - 1, & 2 \leq i \leq \min(A, B) \\ \min(A, B), & \min(A, B) < i \leq \max(A, B) \\ A + B + 1 - i, & \max(A, B) < i \leq A + B \end{cases}$$
 的贡献。

计算贡献

这样的折线，可以通过二维前缀和差分来做到 $O(1)$ 加入，最后 $O(n)$ 统一处理。对于前两类，分别只有 $O(m)$ 对，故依次处理它们的时间复杂度是可以接受的。但是对于第三类，一共会有 $O(m^2)$ 对，直接处理的复杂度是无法接受的。

注意到记 t_i 为相邻两个机场的编号差（认为 0 和 $n+1$ 也是机场），我们有

$$\sum t_i = n。$$

而第三类，刚好就是对所有 $t_i, t_j, i < j$ 进行统计，故 t 之间的相对顺序是不重要的。

而 $\sum t_i = n$ ，则本质不同的 t 只有 $O(\sqrt{n})$ 种，对每一对本质不同的 t_i, t_j 统一计算贡献，故只需要考虑 $O((\sqrt{n})^2) = O(n)$ 对。

最终总时间复杂度为 $O(n+m)$ ，可以通过。

简要题意

Strange Sorting 2

给定一颗有 n 个节点的树 T 。对于一个序列 a ，你可以对其进行若干次操作：选择两个数 a_i 和 a_j ，要求其在 T 上有边直接连接，然后交换 a_i 和 a_j 的值。对于 $l = 1, 2, \dots, m$ ，问有多少个长度为 l ，由 $1 \sim n$ 的序列，可以被上述操作排序，答案对 $10^9 + 7$ 取模。

$1 \leq n \leq 200$, $1 \leq m \leq 10^5$ 。2s, 1G。

Hint

- 特殊化问题：什么样的**排列**可以被排序。
- 如何转化成树上计数。
- m 很大, n 很小。

分析题意

对于任何一个排列 p , 我们先选择树上的一个叶子 v , 我们可以通过若干次操作将 v 移动到第 v 位, 然后就可以不管这个数。在树上将这个叶子去掉, 就可以递归子问题了。

因此, 任意的排列都是可以被排序的。

更进一步的分析, 如果树上中只保留 l 里出现了的数对应的节点, 则可以每一个连通块内的数可以任意交换。

统计答案

我们不妨枚举这样的连通块，则对于块内，需要统计“长度为 l 的序列，每一个数至少出现一次”的问题。对于块间，由于我们需要其能够被排序，所以其组合方式是**唯一的**。

考虑答案的生成函数，就应当是类似于：

$$\sum_{\text{连通块划分}} \prod_{\text{连通块 } \mathcal{C}} \sum_{t=0}^{+\infty} |\mathcal{C}|! S(t, |\mathcal{C}|) x^t$$

其中 S 为第二类斯特林数。

计算答案

记 $F_i(x) = \sum_{t=0}^{+\infty} i! S(t, i) x^t$, 那么答案实际上就是要做类似于树上背包的问题。

即 $f_{i,j}$ 表示处理了 i 在子树内, 当前 i 所在的连通块大小为 j ($j = 0$ 表示 i 未出现) 的方案数 (不考虑 i 所在的连通块) 的生成函数。

初值为 $f_{i,0} = f_{i,1} = 1$ 。

合并 i 的一个儿子 v 的时候, 转移为:

$$f_{i,0} \leftarrow f_{i,0} \times \left(\sum_{l=0}^m f_{v,l} F_l(x) \right), \quad f_{i,l} \leftarrow \sum_{k=0}^l f_{i,k} \times f_{v,l-k}.$$

计算答案

将第二类斯特林数展开，可以得到：
$$F_i(x) = \sum_{j=0}^i (-1)^{i-j} \binom{i}{j} \frac{1}{1-jx}.$$

分式并不是我们喜欢的东西。

注意到 $F_i(x)$ 的分母为 $\prod_{j=1}^i (1-jx)$ 。而所有连通块的大小之和为 n 。

故最终答案中分母 $1-jx$ 项的系数最多为 $\left\lfloor \frac{n}{j} \right\rfloor$ 次。即最终的分子和分母最多只有 $O(n \ln n)$ 次。

计算答案

将分母当作最大的可能分母维护，故只需要考虑维护分子。

如果使用暴力卷积维护分子的时间复杂度为 $O(n^4 \log n)$ ，无法通过。

考虑对 DP 时对分子维护插值，可以将时间复杂度优化至 $O(n^3 \log n)$ 。使用 $O(n^2 \log^2 n)$ 的时间反解出多项式即可。

分母的部分可以 $O(mn \log n)$ 的递推出结果。

最终时间复杂度为 $O(n^3 \log n + n^2 \log^2 n + mn \log n)$ ，可以通过。

Q&A

谢谢大家。